

Preliminary Design Review & Prototype Multi-Author Writing Style Change Detection

Tshephang Phuti-A-Nkutu Matlala¹

Academy of Computer Science and Software Engineering,
University of Johannesburg
223004635@student.uj.ac.za

1 Conceptual Class Diagram (Domain Model)

The domain model in Figure 1 represents the core entities and relationships associated with the multi-author writing style change detection system. The diagram is intentionally restricted to problem-domain concepts and excludes implementation-specific constructs such as sentence buffers, feature vectors, or prediction caches. This abstraction level aligns with conceptual modelling conventions, where design and implementation decisions are deferred to subsequent phases of development.

1.1 Justification

- **User** – represents an individual who submits text for analysis and reviews author-switch prediction results.
- **Admin** – represents a user with elevated privileges responsible for model training, evaluation, and model comparison activities. At the conceptual level, administrative responsibilities are modelled separately from standard user interactions without implying implementation inheritance.
- **Data** – stores uploaded or pasted text together with metadata such as document size, sentence count, and upload format.
- **Features** – stores the stylometric feature vectors extracted from each sentence pair, serving as the input representation for the classifier.
- **Model** – represents a trained classification model, including its configuration parameters and evaluation results.
- **Evaluation** – aggregates cross-validation metrics, confusion matrices, and comparative performance statistics used during model assessment.

The diagram deliberately omits a direct structural association between **Admin** and **Model** because the relationship is behavioural rather than persistent in nature. This interaction is represented as a **«use»** dependency within the design-level model rather than within the conceptual domain model.

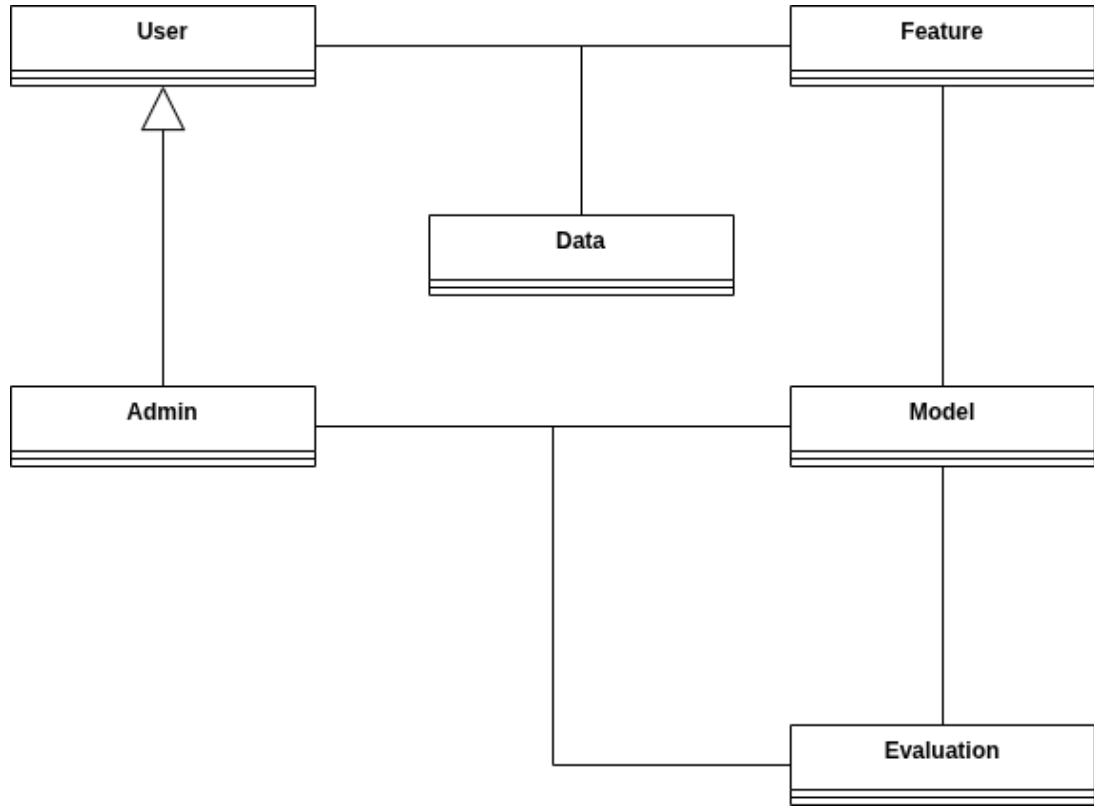


Fig. 1: Conceptual class diagram for the multi-author writing style change detection system. The diagram excludes implementation-level constructs and focuses on persistent domain entities.

1.2 Use Case Traceability

Table 1 establishes traceability between the primary use cases, functional requirements, validating test cases, and the current prototype implementation status.

Table 1: Use Case Traceability

Use Case	Related FRs	Validating Tests	Prototype Status
UC1: Upload text for analysis	FR1–FR3	UT-06–11, IT-01–03	Implemented
UC2: View author-switch predictions	FR5–FR6	IT-04–06, ST-01–02	Implemented
UC3: Interpret confidence scores	FR7	ST-01, UAT	Partial
UC4: Train classification model	FR8	IT-07–08, ST-04	Implemented
UC5: Compare model performance	FR9	IT-11	Partial
UC6: Login and authentication	FR9 (implied)	UT-01–05	Partial

2 System Architecture Justification

The system architecture is organised into four high-level packages: **Data**, **Features**, **Model**, and **Evaluation**. Each package corresponds to a distinct stage of the author-switch detection pipeline and encapsulates a clearly defined set of responsibilities.

This separation of concerns provides the following benefits:

- Modifications to sentence segmentation or input validation are isolated within the **Data** package;
- Additional stylometric features may be introduced within the **Features** package without affecting other components;
- The classification algorithm may be replaced or extended independently within the **Model** package;
- Evaluation metrics and reporting functionality may evolve independently within the **Evaluation** package;
- The modular structure improves maintainability and simplifies unit and integration testing.

The four-package decomposition aligns directly with the conceptual class diagram presented in Section 1, ensuring that each functional requirement is allocated to a well-defined and testable subsystem.

3 Preliminary Test Plan

This preliminary test plan defines the testing strategy, levels, test cases, environment, and success criteria for the multi-author writing style change detection system. Testing follows the test pyramid principle, where a large base of unit tests supports smaller layers of integration and system tests [1].

In scope: functional correctness of text ingestion, feature extraction, prediction, authentication, and result display; classifier performance against baseline; usability of the interface.

Out of scope: load and stress testing beyond the 50-sentence performance requirement; security penetration testing; browser compatibility beyond Chrome and Firefox.

3.1 Test Objectives

1. Each functional requirement is validated against its corresponding specification.
2. The classifier demonstrates improved macro-averaged F1 performance relative to a simple baseline classifier on the held-out PAN 2026 dataset split [2].
3. Documents containing up to fifty sentences are processed within a time frame suitable for interactive use.
4. The system correctly displays the stylometric features most strongly associated with predicted author-switch positions.
5. Invalid, incomplete, or empty inputs are rejected with informative error messages while maintaining system stability.
6. Authentication mechanisms correctly validate or reject login attempts based on credential correctness.

3.2 Test Strategy

Approach. A white-box approach is employed for unit and integration tests, where internal module interfaces are directly exercised. Black-box testing is used for system and acceptance tests, where only external HTTP and UI interfaces are exercised.

Framework. All automated tests are written with `pytest` using the standard-library `unittest.mock`. The Flask test client simulates HTTP requests without starting a live server, enabling fast, deterministic integration tests.

Test Data.

- **Unit tests:** hand-crafted sentences with known feature values computed manually.
- **Integration and system tests:** synthetic dataset with ground-truth author-switch labels.
- **Model evaluation:** PAN 2026 held-out test split.

3.3 Test Levels

Unit Testing Unit testing validates each module in isolation. Target statement coverage is $\geq 80\%$. Components under test include sentence splitting, feature extraction, pairwise differencing, authentication validation, and file size checking.

Integration Testing Key integration scenarios executed through the Flask test client include: document upload and inspection, prediction with and without a trained model, training pipeline execution, model comparison, and login validation.

System Testing End-to-end workflows that exercise the full stack: valid multi-author document processing, single-author document (zero expected switches), empty submission rejection, and fifty-sentence document processing within acceptable time bounds.

User Acceptance Testing (UAT) A proxy user attempts the upload-analyse-review workflow without prior training. The observer records task completion time, error counts, and subjective usability ratings on a 5-point Likert scale.

3.4 FR–Test Case Traceability

Table 2 maps every functional requirement to the test cases that exercise it.

Table 2: FR–Test Case Traceability

FR	Requirement	Covering Test Cases
FR1	Upload or paste text	UT-06–09, IT-01–02, ST-01
FR2	Sentence segmentation	UT-10–11
FR3	Input validation	UT-06, UT-08–09, UT-11, IT-03
FR4	Feature extraction	UT-12–16
FR5	Pairwise differences	UT-16
FR6	Author-switch prediction	UT-17–20, IT-04–06, ST-01–02
FR7	Confidence and explanation	ST-01, UAT
FR8	Model training	UT-21, IT-07–08, ST-04
FR9	Model comparison	UT-23, IT-11
Auth	Login authentication	UT-01–05

3.5 Preliminary Test Cases

Table 3: Preliminary Test Cases

ID	Description	Input / Condition	Expected Result
UT-01	Username validation – valid	Existing username entered	Username accepted
UT-02	Username validation – invalid	Non-existent username	Error message displayed
UT-03	Password validation – correct	Correct password entered	Login successful
UT-04	Password validation – incorrect	Invalid password entered	Login rejected
UT-05	Empty login fields	Missing username or password	Validation error displayed

ID	Description	Input / Condition	Expected Result
UT-06	Empty document upload	Empty text submitted	Upload rejected with warning
UT-07	Valid text upload	Multi-sentence document	Document processed successfully
UT-08	Unsupported file format	PDF file uploaded	Upload rejected with error message
UT-09	Oversized document	File exceeding size limit	Upload blocked
UT-10	Sentence segmentation	Multi-sentence paragraph	Sentences segmented correctly
UT-11	Single-sentence submission	One sentence only	Warning displayed
UT-12	Word count extraction	Normal sentence	Count calculated correctly
UT-13	Punctuation density	Sentence with punctuation	Density calculated correctly
UT-14	Function-word ratio	Sentence with known ratios	Ratio computed correctly
UT-15	Type-token ratio	Sentence with unique words	Correct lexical richness value
UT-16	Feature vector generation	Valid sentence pair	Vector generated successfully
UT-17	Prediction without model	Prediction before training	Informative warning displayed
UT-18	Prediction with trained model	Valid model loaded	Predictions generated
UT-19	Confidence-score generation	Prediction executed	Confidence score displayed
UT-20	Author-switch detection	Multi-author text	Switch positions identified
UT-21	Model training execution	Valid dataset available	Training completes successfully
UT-22	Evaluation metrics	After training	Precision, recall, F1 displayed
UT-23	Model comparison	Two trained models	Comparison table displayed correctly

ID	Description	Input / Condition	Expected Result
IT-01	Inspect endpoint – valid text	POST /inspect with 5-sentence text	HTTP 200; sentence cards displayed
IT-02	Inspect endpoint – file upload	POST /inspect with .txt attachment	HTTP 200; content parsed correctly
IT-03	Inspect endpoint – empty body	POST /inspect with empty textarea	Flash warning displayed
IT-04	Predict endpoint – no model	POST /predict without trained model	Message: “No model available”
IT-05	Predict endpoint – with model	POST /predict after training	HTTP 200; predictions shown
IT-06	Predict endpoint – fewer than 2 sentences	POST /predict with one sentence	Flash warning displayed
IT-07	Train endpoint – dataset missing	POST /train with empty directory	Flash error; log shows dataset error
IT-08	Train endpoint – full pipeline	POST /train with valid dataset	model.pkl created on disk
IT-09	Explore endpoint – statistics	GET /explore with valid dataset	Statistics correct per difficulty level
IT-10	Explore endpoint – both example types	GET /explore with mixed labels	Both columns displayed correctly
IT-11	Model comparison endpoint	POST /compare with two model IDs	Side-by-side metrics displayed
ST-01	End-to-end – two known switches	10-sentence document with ground truth	Both switches detected; top-3 features shown
ST-02	End-to-end – single-author	10-sentence same-author document	Zero switches detected
ST-03	Performance – 50 sentences	50-sentence document submitted	Processing completes within acceptable time
ST-04	End-to-end – train then predict	Full workflow on dataset	Model mode used; F1 displayed

3.6 Test Environment

Table 4: Test Environment

Component	Details
Laptop Model	Acer Aspire A315-58
Operating System	Fedora Linux 44 (Workstation Edition)
Processor	11th Gen Intel Core i3-1115G4
Memory	8 GB RAM
Python Version	3.10+
Framework	Flask
ML Library	scikit-learn
Testing Framework	pytest
Browsers	Chrome 124+, Firefox 125+

3.7 Risks and Contingency

Table 5: Test Risks and Mitigations

Risk	Likelihood	Mitigation
PAN dataset unavailable	Medium	Use synthetic dataset as drop-in replacement
Classifier performance below baseline	Medium	Ablation study; add features if needed
Time constraints limit UAT participants	High	One proxy instructor sufficient; structured walkthrough
Training too slow for CI pipeline	Low	Mock classifier in integration tests; nightly full training

3.8 Acceptance Criteria

The system is considered ready for the subsequent development phase when the following conditions are satisfied:

- All unit tests execute successfully with statement coverage meeting or exceeding 80% for core processing modules;
- All documented integration scenarios execute without runtime errors;

- Preliminary classifier evaluation demonstrates performance exceeding a simple baseline classifier on the held-out PAN test split;
- A fifty-sentence document is processed within a time frame suitable for interactive use (target: under ten seconds on reference hardware);
- The UAT proxy user completes the upload-analyse-review workflow without external assistance and provides a usability rating of at least 4 on a 5-point Likert scale.

4 proof-of-concept prototype

4.1 Prototype Scope and Purpose

The prototype is implemented as a Flask-based web application intended to validate two key aspects of the proposed system design:

1. the feasibility of the proposed stylometric feature extraction and author-switch detection pipeline; and
2. the usability of the user interaction and result presentation workflow.

The prototype intentionally excludes functionality that depends on complete dataset availability, extensive hyper-parameter optimisation, or production-level deployment infrastructure.

4.2 Prototype Overview

The current prototype demonstrates the end-to-end workflow of the multi-author writing style change detection system, including text submission, feature extraction, model training, prediction, and result visualisation. The implementation focuses on validating architectural feasibility and user interaction patterns rather than achieving final classifier performance.

Essential distinction: This prototype serves as a proof-of-concept implementation intended to validate the core processing pipeline and user interaction workflow. It does not constitute a complete implementation of the final system described in the analysis and design artefacts.

4.3 Running the Prototype

1. Install dependencies: `pip install flask numpy scikit-learn`
2. Start the server: `python app.py`
3. Navigate to `http://localhost:5000` in a browser.

Authentication is partially implemented in the prototype; full credential validation is scheduled for the final implementation.

4.4 Implemented Functionality

- **Inspect page (FR1–FR4):** text upload, sentence segmentation, 31 stylometric features per sentence, overview statistics, and colour-coded style-distance bar.
- **Explore page:** dataset statistics (switch rate, sentence count) and example display.
- **Train page (FR8):** classifier selection (Logistic Regression, SVM, k-NN, MLP), cross-validation, synchronous training with live log streaming.
- **Predict page (FR5–FR7):** text submission, probability threshold slider, ML-based predictions when a model exists.
- **Results page:** held-out F1, precision, recall, and cross-validation statistics.
- **Authentication (partial):** login form and session management; full validation deferred.

4.5 Functional Requirement Coverage

Table 6: Prototype Functional Requirement Coverage Compared to the Final Implementation.

FR	Requirement	Prototype	Final System
FR1	Upload or paste text	Implemented	Implemented
FR2	Sentence segmentation	Implemented	Implemented
FR3	Input validation and errors	Implemented	Implemented
FR4	Stylometric feature extraction	Implemented	Implemented (additional features)
FR5	Pairwise feature differences	Implemented	Implemented
FR6	Author-switch prediction	Implemented	Implemented
FR7	Confidence and top features	Partial	Implemented
FR8	Admin model training	Implemented	Implemented
FR9	Model comparison	Partial	Implemented
Auth	User authentication	Partial	Implemented

4.6 Known Limitations and Justifications

- **Random-baseline predictor:** The prototype includes a random prediction fallback when no trained model is available. This exists solely for interface exploration. The final implementation will require a trained model before prediction is permitted.
- **Synchronous training:** Training blocks the HTTP response. For large datasets, this may exceed timeout limits. The final implementation will delegate training to a background task queue.
- **Upload format restriction:** Only .txt files with a 2 MB limit are accepted. PDF and DOCX support is planned for the final system.
- **Top feature computation:** Currently extracted from raw feature difference vectors. Full permutation importance ranking is deferred until the final implementation.

4.7 Prototype Screenshots

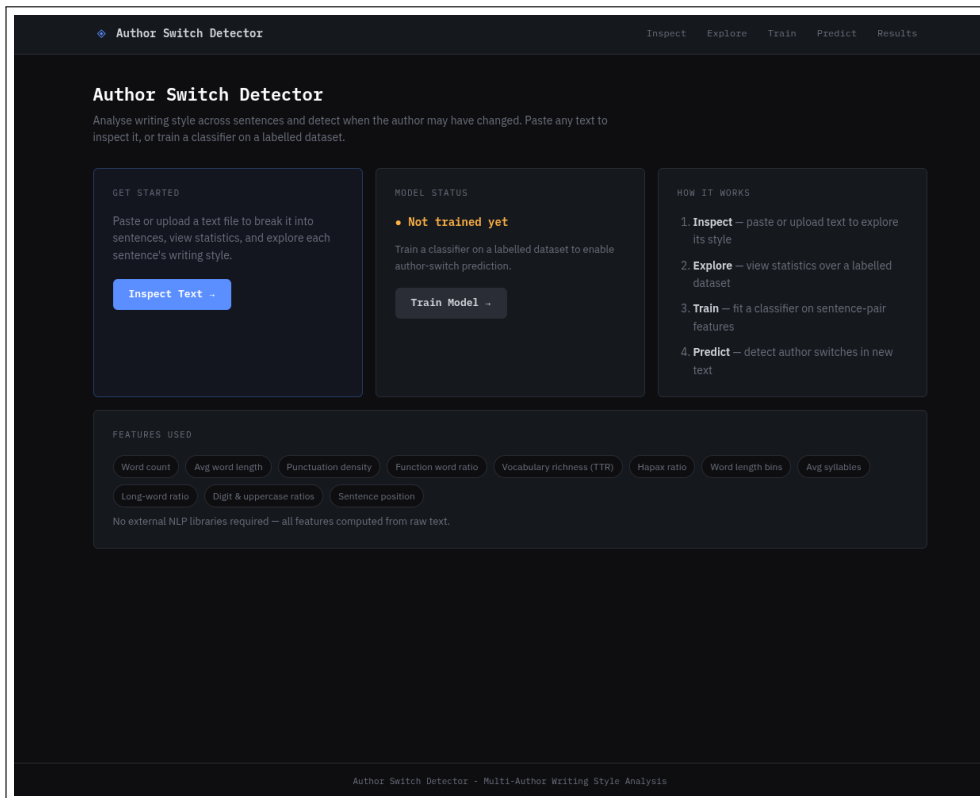


Fig. 2: Prototype home page displaying model status, quick-start links, and stylometric feature groups.

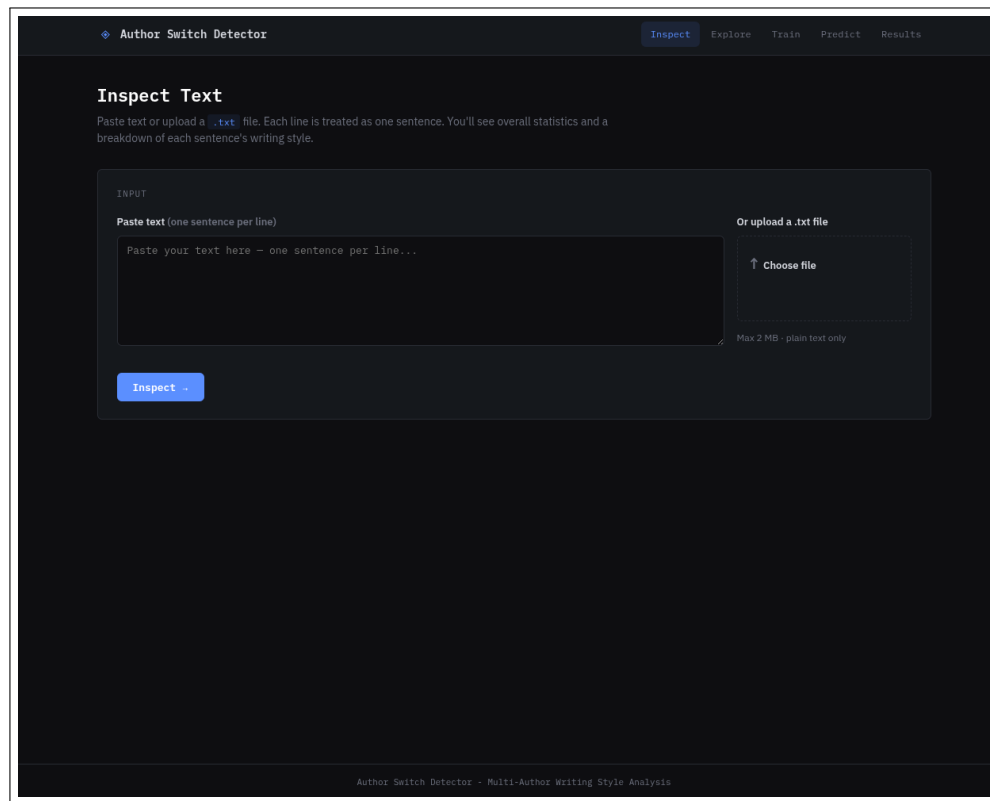


Fig. 3: Inspect page showing per-sentence feature chips and colour-coded style-distance bars between consecutive sentences. Red borders indicate high stylistic distance.

Author Switch Detector

Inspect Explore Train **Predict** Results

Predict Author Switches

Paste text or upload a [.txt](#) file. The model will label each consecutive sentence pair as *same author* or *author switch*.

No trained model found — running in random mode. Each pair is assigned a random probability. This is a useful baseline: any trained model should do better than this. [Train a model --](#)

INPUT

Paste text (one sentence per line)

Paste sentences here, one per line...

Or upload a .txt file

↑ Choose file

Decision threshold: 0.50

Lower -- more switches flagged · Higher -- fewer

Run Detection Inspect First --

RESULTS

Results will appear here once you submit text.

Author Switch Detector - Multi-Author Writing Style Analysis

Fig. 4: Predict page displaying author-switch analysis results with confidence scores and explanatory features.

References

1. Myers, G.J., Badgett, T., Thomas, T.M., Sandler, C.: The art of software testing, vol. 2. Wiley Online Library (2004)
2. Zangerle, E., Mayerl, M., Stein, B., Potthast, M.: Pan26 multi-author writing style analysis (Mar 2026). <https://doi.org/10.5281/zenodo.19068843>, <https://doi.org/10.5281/zenodo.19068843>